

Production Grade APIs PART ②

Date |
Page | 24

High Level Flow

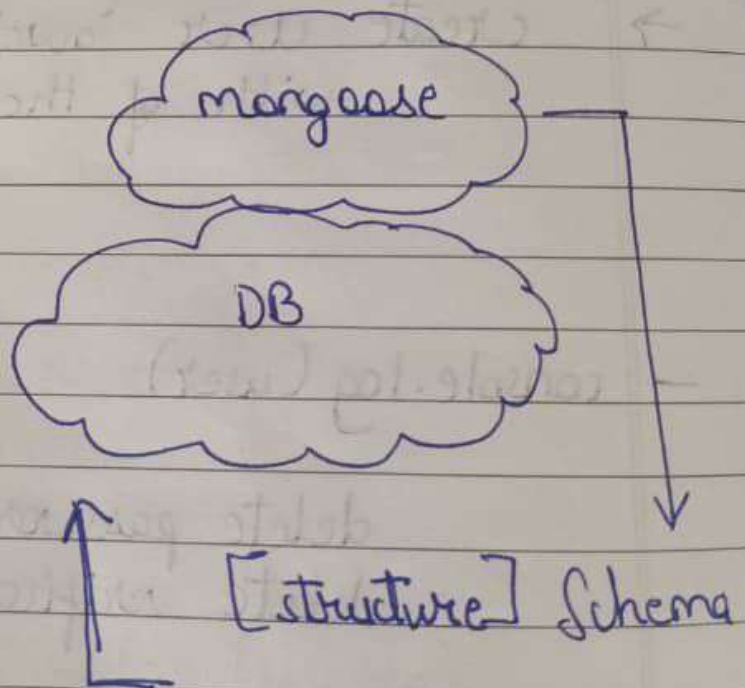
└ user → server

↓
fetch() {
 name;
 email;
 password;
}

Into the Auth Module

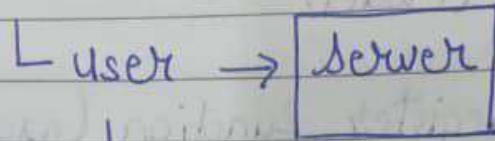
→ '/register'

→ controller
└ services



→ To ensure the data authenticity of data we use (DTO)

High Level Flow



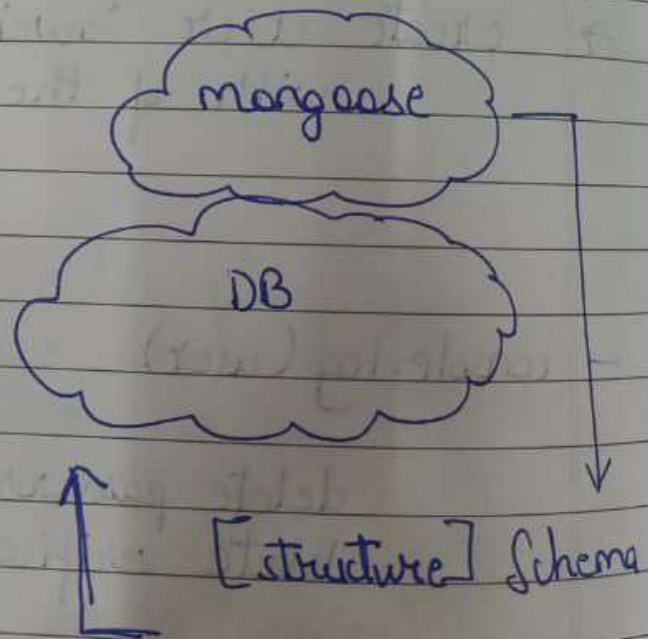
```

fetch() {
  name,
  email,
  password
}
    
```

Into the Auth Module

→ `/register`

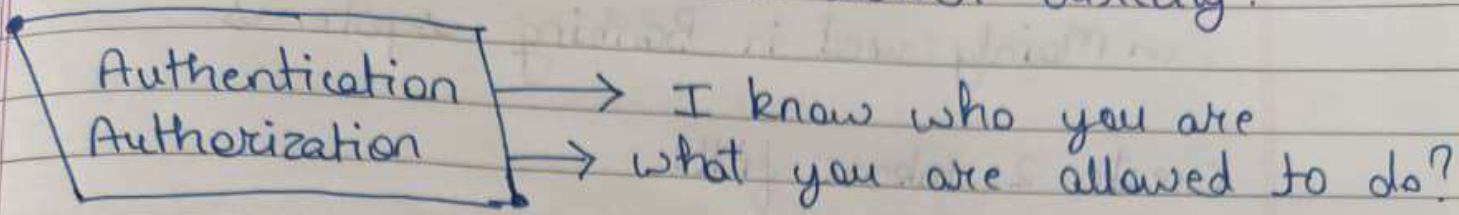
→ controller
 └ services



→ To ensure the data authenticity of data we use (DTO)

Types {},

→ client cannot talk to the server directly.

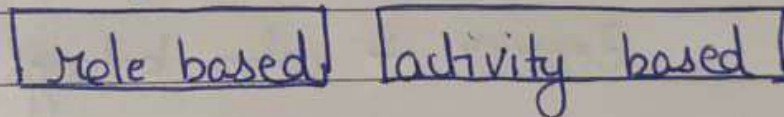


In Broader perspective -

Authentication is like an (admit card).

This means we are your authentic user!

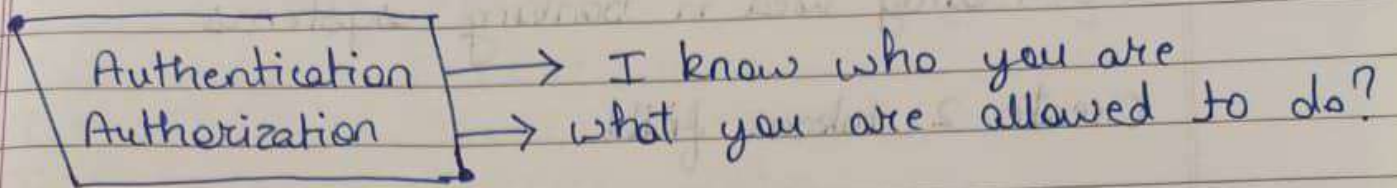
Authorization on the other hand is what we are allowed to do.



Session Based authentication

→ store a session with a boolean value.
- play on it.

→ client cannot talk to the server directly.

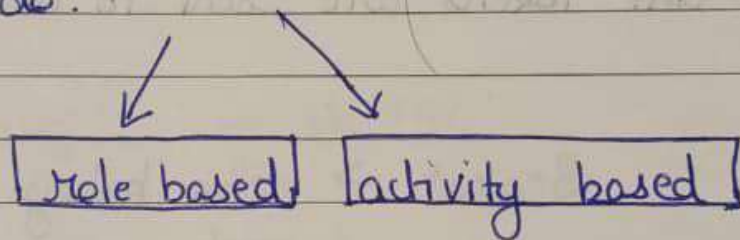


In Broader perspective -

Authentication is like an (admit card).

This means we are your authentic user!

Authorization on the other hand is what we are allowed to do.



Session Based authentication

→ store a session with a boolean value.

-play on it.

Date |
Page | 28
15/20min short Lived Token/sessions

↳ Mainly used in Banking systems

Create 2 tokens field

- Access token (15 min)

- Refresh Token (24 hrs)

Token (JWT)

⇒ Format - xxxx.yyyy.zzzz (Header.Payload.Signature)

Both the tokens are sent to db and client

Bearer → who belongs the token?

Examples:

PVR/Cinema Hall ticket: scans the QR on the ticket
(verify)
from the database

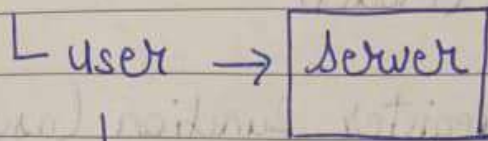
Production Grade APIs PART ②

Date |

Page |

24

High Level Flow

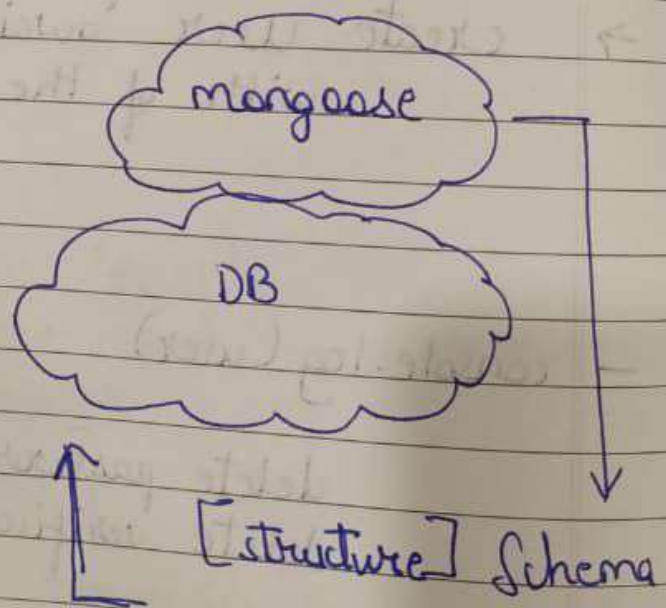


```
fetch() {  
  name,  
  email,  
  password  
}
```

Into the Auth Module

→ '/register'

→ controller
└ services



→ To ensure the data authenticity of data we use (DTO)

Types { }

Env File

Date |

Page | 27

- JWT_ACCESS_SECRET: <any strong string>
- JWT_ACCESS_EXPIRY: 15min
- JWT_REFRESH_SECRET: <string>
- JWT_REFRESH_EXPIRY: 24hr

Encryption & Decryption

Encrypt can be decrypt

123 → abc → 123

Hashing

- sha256

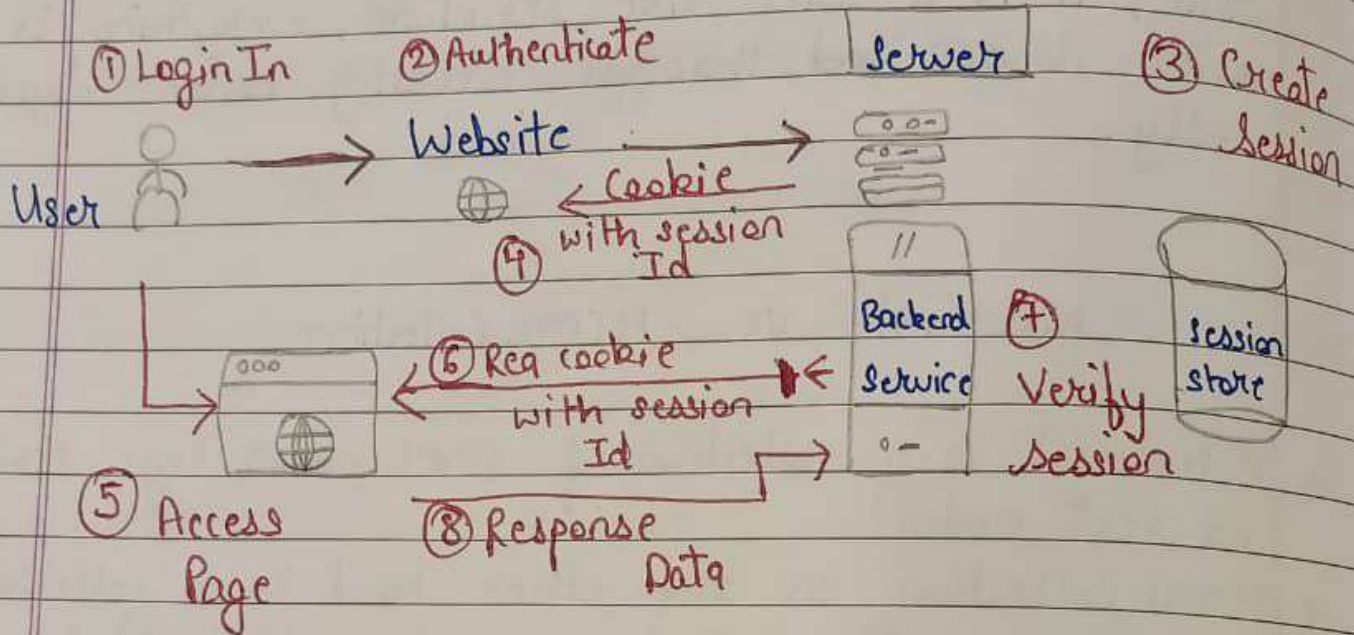
It can never be undone!

123 → 12234gh/cD1

Catch!

123 will result everytime same hash

Session Based Authentication



Creating Login Service

- create an async function (login)

take email and password from the params

check User → case sensitive

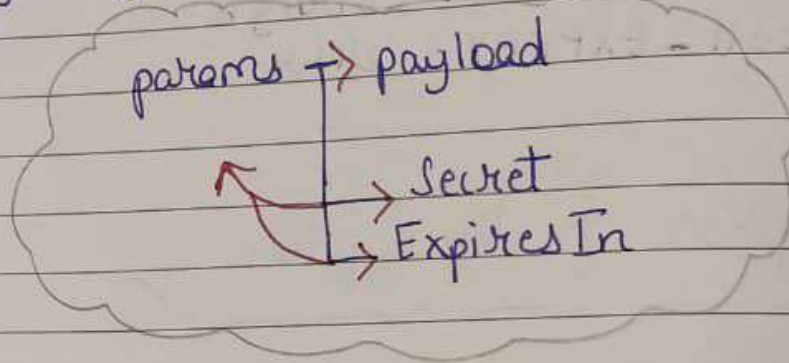
X user

check the user from the database with email and select password

- await the User and hold it in 'user' variable

Generate Access Token

⇒ 'jwt.sign'



Generate Refresh Token

same

⇒ Verify & Generate Access Token

X - [~~Generate Verify Token~~: Generate AccessToken]

→ Again take payload as in params

create a function 'verifyAccessToken'

- take token as params
- use 'jwt.verify'

⇒ 'GenerateAccessToken'

- same,

payload in params

→ (jwt.sign)

39
user.toObject() to make the user an object

- delete password
- delete refreshToken

⇒ return user as an object with
{ userObj, accessToken, refresh-
-hToken

Refresh token

- async function, token as params/in
- firstly, validate that if token does not exists throw standardized unauthorized error ("Refresh Token missing")
- verify the refresh using 'verifyRefreshToken' utility
- take it in 'decoded' variable

Flow → 1. Find user

2. If user not found throw error rel.

3. If user exists; but it's refresh token does not matches with the same hashed token throw - Invalid token error

4. else or /

→ GenerateAccessToken()

return accessToken.

✓ If user does not exist throw standardized error response

('Invalid email and password')

and if user is not verified throw forbidden error response.

Tokens → accessToken
→ refreshToken (hashed)

✓ Try to keep the token light-weight.

creating a hashing utility
'hashToken'

- token in params
- hash the token using crypto ('sha256')

save hashed refreshToken in user

save the user

→ set validationBeforeSave to - false

A validation is performed again when saving the user

- await the user